

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №1

з дисципліни

«Об'єктно-орієнтоване програмування»

на тему

«Знайомство із середовищем розробки та складання модульних проєктів програм»

Виконала:

студентка групи ІМ-55
Сітковська Адіна Олександрівна
Номер в списку - 27

Перевірив:

Рекечинський Д. О.

Завдання:

1. Створити у обраному середовищі розробки проєкт з ім'ям Lab1.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.

Завдання згідно варіанту:

B1 = 3: Вікно діалогу з елементом списку (List Box) та двома кнопками: [Так] і [Відміна]. У список автоматично записуються назви груп нашого факультету. Якщо вибрати потрібний рядок списку і натиснути [Так], то у головному вікні повинен відображатися текст вибраного рядка списку.

B2 = 0: Вікно діалогу для вводу тексту, яке має стрічку вводу (Edit Control) та дві кнопки: [Так] і [Відміна]. Якщо ввести рядок тексту і натиснути [Так], то у головному вікні повинен відображатися текст, що був введений.

Для усіх варіантів необхідно створити два пункти меню — "Робота1" та "Робота2".

1. Потрібно, щоб при виборі пункту меню "Робота1" виконувалося щось згідно варіанту B1.
2. Запрограмувати також, щоб при виборі пункту меню "Робота2" виконувалося щось згідно варіанту B2.
3. Вихідний текст функції B1 запрограмувати в окремому модулі, а вихідний текст для функції B2 — у іншому окремому модулі. Таким чином, проєкт повинен складатися з головного файлу та ще, як мінімум, двох файлів модулів.
4. Callback-функції вікон діалогу не мають бути видимими зовні модулів. Це обмеження пов'язане з тим, що доступ до логіки окремого діалогу повинен мати лише сам діалог. Крім цього, не можна виносити за межі модулів ресурси (опис вікон діалогу).
5. Якщо виводиться наступне вікно діалогу, то попереднє повинне закриватись автоматично.
6. Не допускається імпорт коду для модулю з іншого модулю. Підказка: якщо є спільна логіка для модулів, яку не хотілось би дублювати, то краще її винести в окремий файл (utils).

Код:

main.py:

```
import tkinter as tk
from tkinter import ttk
from module1 import show_work1
from module2 import show_work2

def handle_work1():
    value = show_work1(root)

    if value is not None:
        message_var.set(value)

def handle_work2():
    value = show_work2(root)

    if value is not None:
        message_var.set(value)

root = tk.Tk()
root.title("lab1")
root.geometry("600x380")

style = ttk.Style()
style.theme_use("clam")

menu_bar = tk.Menu(root)

actions_menu = tk.Menu(menu_bar, tearoff=0)
actions_menu.add_command(label="Робота1", command=handle_work1)
actions_menu.add_command(label="Робота2", command=handle_work2)
menu_bar.add_cascade(label="Дії", menu=actions_menu)

root.config(menu=menu_bar)

message_var = tk.StringVar()
ttk.Label(root, textvariable=message_var, font=("Arial", 14, "bold")).pack(pady=170)

root.mainloop()
```

module1.py:

```
import tkinter as tk
from tkinter import ttk
from utils import create_dialog, close_dialog

GROUPS = [
    "ІП-31", "ІП-32", "ІП-33",
    "ІО-31", "ІО-32",
    "СП-31", "СП-32",
    "ТП-31", "ТП-32",
]

def show_work1(parent):
    dialog = create_dialog(parent, "Робота1")
    result = [None]
```

```

listbox = tk.Listbox(dialog, height=8, width=20, exportselection=False)
for group in GROUPS:
    listBox.insert(tk.END, group)
listbox.select_set(0)
listbox.pack(padx=16, pady=(16, 8))

buttons = ttk.Frame(dialog)
buttons.pack(pady=(0, 16))

def on_ok():
    selection = listBox.curselection()
    value = listBox.get(selection[0]) if selection else None
    close_dialog(dialog, result, value)

ttk.Button(buttons, text="Так", width=10, command=on_ok).pack(side="left", padx=4)

ttk.Button(buttons, text="Відміна", width=10, command=lambda: close_dialog(dialog, result,
None)).pack(side="left", padx=4)

dialog.wait_window()
return result[0]

```

module2.py:

```

import tkinter as tk
from tkinter import ttk
from utils import create_dialog, close_dialog

def show_work2(parent):
    dialog = create_dialog(parent, "Робота2")
    result = [None]

    entry_var = tk.StringVar()
    entry = ttk.Entry(dialog, textvariable=entry_var, width=30)
    entry.pack(padx=16, pady=16)
    entry.focus_set()

    buttons = ttk.Frame(dialog)
    buttons.pack(pady=(0, 16))

    ttk.Button(buttons, text="Так", width=10, command=lambda: close_dialog(dialog, result,
entry_var.get())).pack(side="left", padx=4)

    ttk.Button(buttons, text="Відміна", width=10, command=lambda: close_dialog(dialog, result,
None)).pack(side="left", padx=4)

    dialog.wait_window()
    return result[0]

```

utils.py:

```

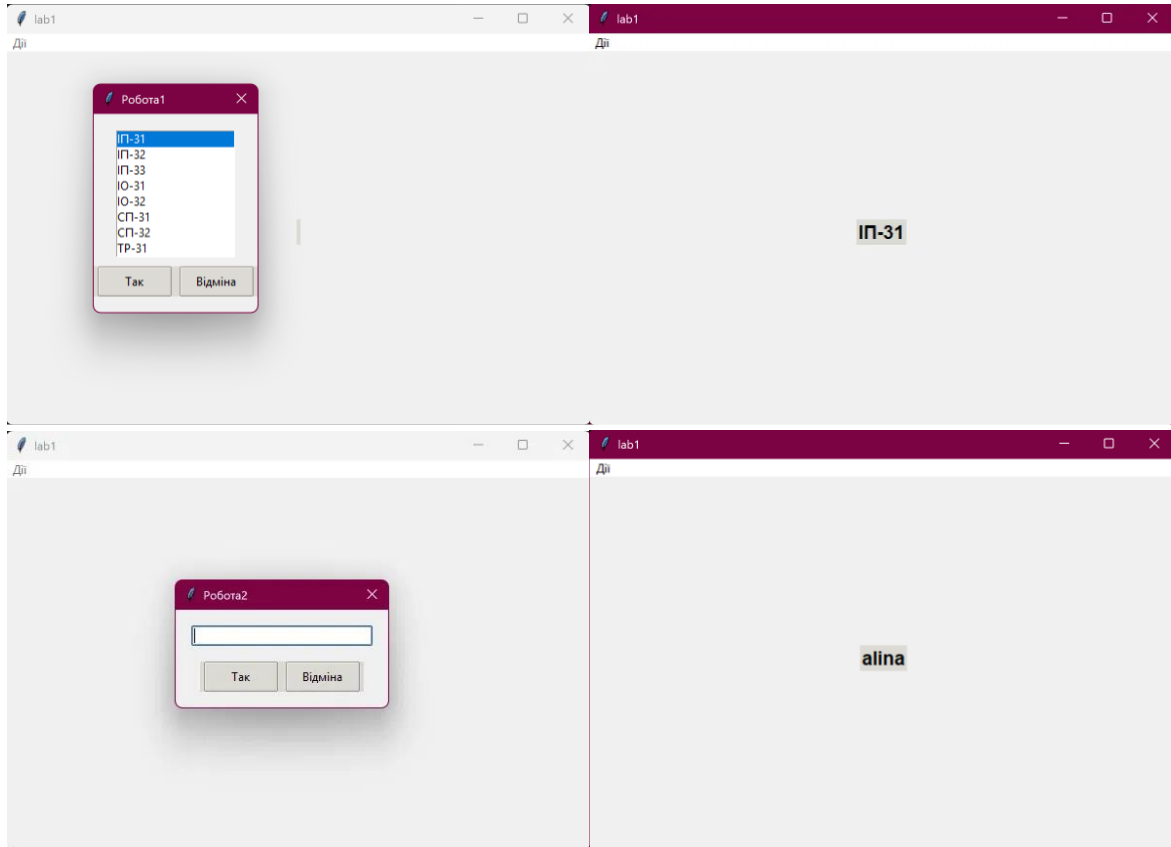
import tkinter as tk

def create_dialog(parent, title):
    dialog = tk.Toplevel(parent)
    dialog.title(title)
    dialog.resizable(False, False)
    dialog.transient(parent)
    dialog.grab_set()
    return dialog

```

```
def close_dialog(dialog, result_holder, value):  
    result_holder[0] = value  
    dialog.destroy()
```

Скріншоти:



Висновок

На даній лабораторній роботі я отримала перші навички розробки програм з графічним інтерфейсом та ознайомилась з принципами модульного програмування на прикладі Python. У роботі реалізовано два сценарії: "Робота1" — вибір групи факультету зі списку (Listbox), "Робота2" — введення довільного тексту через поле Entry. Спільна логіка створення і закриття діалогового вікна винесена в окремий файл `utils.py`, а кожен з

модулів `module1.py` та `module2.py` відповідає лише за свій діалог і не залежить від іншого. Головний файл `main.py` викликає обидва модулі лише через одну функцію кожного (`show_work1` / `show_work2`) і нічого не знає про їхню внутрішню будову. Це допомогло краще зрозуміти, як розділяти програму на незалежні частини так, щоб зміни в одному модулі не зачіпали інші.